

Unsloth 와 QLoRA 를 활용한 Qwen2.5-7B 파인튜닝 기반 구조화 고객 리뷰 분석

형식 강제 응답 (항상 JSON, 표 또는 특정 템플릿 형식으로 출력)

짧은 단일 지시문만으로 고객 리뷰를 하나의 정형화된 JSON 객체로 변환하도록 기본 LLM 학습하기

과목: 자연어 정보 분석

이름: 곽가이(JIAYI QU)

학번: 2023106031

제출일: 2026 년 6 월

초록

본 보고서는 하나의 구체적인 공학적 목표를 중심으로 수행한 실습형 파인튜닝 프로젝트를 다룬다. 목표는 단 하나의 짧은 시스템 프롬프트만을 사용하여 대규모 언어 모델이 영어 고객 리뷰를 감성 분류, 점수, 토픽 목록, 그리고 한 문장 요약에 포함하는 고정된 구조의 JSON 객체로 변환하도록 만드는 것이다. 본 과제는 구조화 출력(Structured Output) 문제에 해당하며, 출력 결과의 JSON 유효성 비율을 통해 성능 향상을 시각적으로 확인할 수 있을 뿐만 아니라 정량적으로도 측정할 수 있다.

모든 실험은 단일 16GB RTX 5060 Ti 환경에서 로컬로 수행되었으며, 기본 모델로는 Qwen2.5-7B-Instruct 를 사용하였다. 또한 파라미터 효율적 파인튜닝 기법인 4 비트 QLoRA 를 적용하기 위해 Unsloth 프레임워크를 활용하였다. 학습 데이터는 12 개의 상품 카테고리 및 4 개의 감성 클래스를 포함하는 총 280 개의 예제를 직접 생성하여 구축하였으며, 이를 역할 기반(role-tagged) 채팅 형식으로 구성하였다. 손실 함수는 어시스턴트가 생성하는 JSON 응답 부분에 대해서만 계산하였다.

학습은 총 3 개의 에포크와 105 개의 학습 단계로 진행되었으며, 약 3 분이 소요되었다. 학습 과정에서 손실값은 3.35 에서 0.01 까지 감소하였다. 학습에 사용되지 않은 별도의 테스트 데이터셋에서 평가한 결과, 기본 모델은 8 개의 도메인 리뷰에 대해 유효한 JSON 을 단 한 번도 생성하지 못해 0%의 성공률을 기록하였고, 모든 응답을 일반 문장 형태로 출력하였다. 반면 파인튜닝된 모델은 동일한 테스트에서 100%의 유효 JSON 생성률을 달성하였다.

또한 본 보고서는 파인튜닝 과정에서 나타난 부작용인 형식 전이(format bleeding) 현상도 함께 분석하였다. 이는 모델이 리뷰 분석과 무관한 질문에도 JSON 형식을 강제로 적용하는 현상을 의미한다. 마지막으로 이러한 한계점과 함께 향후 개선 방향에 대해서도 논의하였다.

Keywords:: 대규모 언어 모델 파인튜닝, Unsloth, QLoRA, LoRA, 구조화 출력, Qwen2.5, 파라미터 효율적 파인튜닝.

1 Introduction

1.1 연구 배경 및 동기

고객 리뷰는 제품 개선과 운영 의사결정에 중요한 역할을 한다. 그러나 전자상거래 플랫폼, 앱 스토어, 여행 예약 사이트 등에서 생성되는 방대한 양의 리뷰로 인해 이를 사람이 직접 읽고 분류하며 종합하는 작업은 점점 비현실적인 일이 되었다. 이에 따라 많은 기업과 조직은 이러한 업무를 대규모 언어 모델에 맡기고자 한다.

하지만 모델이 단순히 리뷰의 내용을 자연스러운 문장으로 요약하거나 설명하는 것만으로는 충분하지 않다. 실제 서비스 환경에서는 대시보드, 데이터베이스, 알림 시스템, 자동화 파이프라인 등 다양한 후속 시스템이 활용되기 때문에, 사람이 읽기 좋은 문장이 아니라 프로그램이 직접 처리할 수 있는 구조화된 데이터가 필요하다. 따라서 모델이 모든 리뷰에 대해 동일한 형식의 출력을 안정적으로 생성할 수 있는지가 실제 활용 가능성을 결정하는 핵심 요소가 된다.

최근에는 파라미터 효율적 파인튜닝(Parameter-Efficient Fine-Tuning) 기술의 발전으로 비교적 제한된 하드웨어 환경에서도 대규모 언어 모델을 특정 목적에 맞게 효과적으로 조정할 수 있게 되었다[4]. 본 프로젝트는 이러한 가능성을 바탕으로 진행되었다.

본 연구에서는 문제 범위를 하나의 명확하고 제한된 과제로 좁혀 설정하였다. 즉, 고객 리뷰를 고정된 구조의 JSON 형식으로 변환하는 작업에 집중하고, 이를 중심으로 데이터셋 구축, 모델 파인튜닝, 학습 과정 모니터링, 기본 모델과 파인튜닝 모델의 성능 비교, 그리고 최종 모델 내보내기까지 전체 과정을 수행하였다.

궁극적으로 본 프로젝트의 목표는 단순히 기존 API 를 호출하여 결과를 얻는 데 그치는 것이 아니라, 제한된 연산 자원과 시간 예산 안에서 원시 데이터로부터 실제 배포 가능한 모델을 구축하는 전 과정을 직접 경험하고 검증하는 데 있다.

1.2 과제 정의 및 출력 구조

본 연구에서는 과제를 다음과 같이 정의하였다. 하나의 영어 고객 리뷰가 입력되면 모델은 항상 네 개의 필드를 포함하는 단일 JSON 객체를 출력해야 한다. `sentiment` 필드는 `positive`, `negative`, `neutral`, `mixed` 중 하나의 값만 가질 수 있으며, `score` 필드는 1 부터 5 까지의 정수로 구성된다. `topics` 필드는 리뷰에서 언급된 주요 내용을 요약하는 소문자 형태의 평가 항목 목록이며, `summary` 필드는 한 문장으로 이루어진 짧은 영어 요약문이다. 또한 JSON 객체 이외의 어떠한 추가 텍스트도 허용되지 않는다. 이러한 설계는 성능 평가를 객관적으로 수행할 수 있다는 장점을 가진다. 출력 결과를 파싱한 뒤 필수 필드가 모두 존재하는지, 각 값이 올바른 형식을 따르는지를 확인하면 된다. 이를 통해 유효한 JSON 생성 비율(valid-JSON rate)을 명확하고 일관되게 계산할 수 있다.

구조화 출력 과제를 선택한 또 다른 장점은 파인튜닝의 효과를 매우 직관적으로 확인할 수 있다는 점이다. 기본 모델은 이미 성능이 뛰어난 대화형 모델이지만, 별도의 지시가 없으면 일반적인 챗봇처럼 설명이 포함된 자연어 문장으로 응답하는 경향이 있다. 반면 특정 출력 형식에 맞추어 파인튜닝을 수행하면 고정된 구조를 생성하는 행동이 모델의 가중치에 학습되어 내재화된다. 따라서 기본 모델과 파인튜닝 모델의 출력 결과를 나란히 비교하면 차이를 즉시 확인할 수 있으며, 파인튜닝이 실제로 어떤 변화를 만들어 내는지 효과적으로 보여줄 수 있다.

1.3 평가 전략 및 연구 기여

공정한 비교를 수행하고 파인튜닝의 효과를 보다 명확하게 확인하기 위해, 평가 과정에서는 의도적으로 매우 짧은 시스템 프롬프트만 사용하였다. 또한 해당 프롬프트에는 출력 형식에 대한 어떠한 지시도 포함하지 않았다. 이러한 조건에서 기본 모델은 형식에 대한 단서를 제공받지 못하기 때문에 일반적인 대화형 어시스턴트의 특성에 따라 자연어 설명 형태의 응답을 생성하게 된다. 반면 파인튜닝된 모델은 학습 과정에서 습득한 행동 패턴에 의해 동일한 프롬프트가 주어지더라도 안정적으로 올바른 JSON 형식을 생성할 수 있다.

또한 도메인 리뷰 데이터뿐만 아니라 리뷰 분석과 관련이 없는 일반 질문도 별도로 준비하였다. 이는 파인튜닝 이후 모델이 기존의 일반적인 질의응답 능력을 잃지 않았는지 확인하기 위한 것으로, 흔히 망각(forgetting) 검증이라고 부른다. 이를 통해 모델이 특정 작업을 학습하는 과정에서 다른 능력이 얼마나 유지되는지를 함께 평가하였다.

본 연구의 주요 기여는 다음과 같다. 첫째, 일반 소비자용 하드웨어 환경에서도 재현 가능한 로컬 QLoRA 파인튜닝 파이프라인을 구축하였다. 둘째, 구조화 출력 학습에 적합하도록 설계된 합성 데이터셋을 제작하였다. 셋째, 기본 모델과 파인튜닝 모델을 동일한 조건에서 비교하는 객관적인 평가를 수행하였다. 마지막으로, 파인튜닝 과정에서 발생한 부작용과 한계점을 숨기지 않고 분석하여 향후 개선 방향을 제시하였다.

2 기본 모델 선정

2.1 하드웨어 및 메모리 환경

실험에 사용된 로컬 시스템은 Windows 환경에서 동작하는 단일 NVIDIA GeForce RTX 5060 Ti GPU 를 탑재하고 있으며, 약 15.9GB 의 사용 가능한 GPU 메모리를 제공한다. 해당 GPU 는 Blackwell 아키텍처 기반으로, 연산 능력(compute capability)은 sm_120 이다.

Blackwell 은 비교적 최신 GPU 아키텍처이기 때문에 이를 지원하도록 별도로 컴파일된 CUDA 런타임이 필요하다. 일반적으로 제공되는 PyTorch 버전은 이전 세대 GPU 까지만 지원하는 경우가 많아 해당 그래픽카드를 정상적으로 인식하지 못하였다. GPU 를 정상적으로 사용하기 위해 CUDA 12.x 를 지원하는 PyTorch 버전(실행 환경에서는 버전 2.12.0 으로 표시됨)과 Triton 3.7.0 을 설치하였으며, 이후에야 GPU 가 정상적으로 인식되었다.

사용 가능한 메모리 용량은 모델 선택 과정에서 중요한 제약 조건이 되었다. 4 비트 양자화를 적용할 경우 16GB 메모리 환경에서도 7B 규모의 모델을 무리 없이 실행할 수 있었으며, 학습에 필요한 추가 메모리도 충분히 확보할 수 있었다.

2.2 후보 모델 비교 및 최종 선정

수업에서는 초보자를 대상으로 8B 급 지시형(Instruct) 모델 또는 0.5B~3B 규모의 소형 모델 사용을 권장하였다.

이에 따라 우리는 중국어와 영어 지원 성능, 사고(Thinking) 모드의 존재 여부, 그리고 16GB 환경에서의 실행 가능성을 기준으로 여러 후보 모델을 비교하였다. 비교 결과는 표 1 에 정리하였다.

최종적으로는 Qwen2.5-7B-Instruct 를 기본 모델로 선정하였다[3].

선정 이유는 다음과 같다. 첫째, 영어와 중국어를 모두 우수하게 지원한다. 둘째, 별도의 Thinking 모드를 포함하지 않아 추가적인 추론 과정이 출력에 섞여 들어가지 않으며, 그 결과 더욱 깔끔한 JSON 출력을 생성할 수 있다. 셋째, 4 비트 양자화 적용 후 16GB 메모리 환경에서도 안정적으로 실행할 수 있다. 마지막으로, 관련 커뮤니티와 파인튜닝 자료가 가장 풍부하게 축적되어 있어 예상치 못한 문제 발생 시 해결 방법을 찾기 쉽다는 장점이 있다. 따라서 학습 및 실험 과정의 안정성과 재현성을 고려했을 때 가장 적합한 선택이라고 판단하였다.

Table 1 Comparison of candidate base models

Candidate model	Size	Key consideration for this task	Verdict
Qwen2.5-7B-Instruct	7B	Strong English/Chinese; no thinking mode (clean output); roomy in 16GB after 4-bit; mature ecosystem	Chosen
Qwen2.5-3B-Instruct	3B	Lighter and faster; a viable fallback, but quality slightly below 7B	Backup
Qwen3-4B	4B	Newer, but ships a thinking mode that can leak reasoning text into strict JSON	Not used
Llama-3.1-8B-Instruct	8B	Strong in English but weaker in Chinese; no advantage in this course setting	Not used

2.3 양자화 및 파인튜닝 전략

메모리와 학습 시간의 제약을 고려하여 본 연구에서는 QLoRA 를 적용하였다[2]. 기본 모델은 4 비트 양자화 방식으로 로드한 후 모든 파라미터를 고정하였으며, 그 위에 LoRA(Low-Rank Adaptation) 어댑터를 추가하여 학습을 수행하였다[1]. 이 방식에서는 전체 모델이 아니라 소수의 어댑터 파라미터만 업데이트되므로 학습에 필요한 메모리를 크게 줄

일 수 있다. 동시에 기본 모델이 이미 보유하고 있는 언어 이해 능력과 지식을 유지할 수 있다는 장점도 있다. 사용된 어댑터의 구체적인 하이퍼파라미터와 설정 이유는 4장에서 자세히 설명한다.

2.4 환경 구축 및 엔지니어링 이슈

실제 학습을 시작하기 전에 환경 구축 자체에 상당한 시간이 소요되었으며, 이러한 경험 역시 기록할 가치가 있다고 판단하였다.

처음 사용한 Python 버전은 너무 오래된 버전이었기 때문에 최신 라이브러리들이 문법 호환성 문제로 인해 import 단계에서 오류를 발생시켰다. 이후 새로운 Python 3.11 환경을 구축하자 해당 문제는 즉시 해결되었다.

다음으로, 사용한 GPU가 최신 Blackwell 아키텍처 기반이었기 때문에 기본 설치된 PyTorch는 이전 세대 연산 능력만 지원하도록 컴파일되어 있어 그래픽카드를 인식하지 못했다. 따라서 최신 CUDA를 지원하는 PyTorch 버전을 별도로 설치해야 했다.

설치 순서 또한 매우 중요했다. 먼저 프레임워크를 설치한 뒤 일반적인 PyTorch를 추가로 설치하면 CUDA 지원 버전이 CPU 전용 버전으로 덮어써지는 문제가 발생하였다. 이 경우 프레임워크는 더 이상 GPU를 인식하지 못하게 된다.

따라서 우리는 각 설치 단계를 마칠 때마다 간단한 명령어를 사용하여 GPU가 정상적으로 인식되는지 먼저 확인한 후 다음 단계로 진행하였다. 이를 통해 학습 과정에서 뒤늦게 문제가 발생하는 상황을 방지할 수 있었다.

이러한 과정은 사소해 보일 수 있지만, 실제로는 본 실험을 로컬 환경에서 재현하는 데 있어 가장 큰 장애물이라고 할 수 있다. 모델 학습 자체보다도 환경 설정과 호환성 문제 해결에 더 많은 시간이 소요되는 경우가 많기 때문이다.

3 데이터셋 설계

3.1 출력 구조 정의

데이터셋의 핵심은 각 고객 리뷰와 그에 대응하는 JSON 라벨 간의 일대일 매칭이다. 먼저 출력 구조를 고정하였다: `sentiment`는 네 개의 클래스 중 하나이며, `score`는 1부터 5까지의 정수, `topics`는 평가 항목을 나타내는 단어 리스트, `summary`는 하나의 간결한 영어 문장이다. 학습 데이터의 모든 `assistant` 응답은 이 구조를 엄격하게 따르도록 구성되었으며, 이를 통해 모델이 대량의 일관된 예제를 통해 안정적인 출력 형식을 학습하도록 하였다.

3.2 합성 데이터 구축

사용 가능한 레이블 데이터셋이 존재하지 않았기 때문에, 학습 데이터는 프로그램을 통해 합성하였다. 먼저 무선 이어폰, 노트북, 러닝화, 호텔 숙박, 모바일 앱, 커피머신 등 12개의 제품 카테고리를 사전에 정의하였으며, 각 카테고리마다 관련된 평가 요소(aspects)를 구성하였다. 또한 각 평가 요소에 대해 긍정 및 부정의 자연어 표현을 미리 준비하였다.

각 샘플을 생성할 때는 먼저 감성(sentiment)을 무작위로 결정한 뒤, 이에 따라 적절한 평가 요소들을 선택한다. 이후 선택된 요소들을 기반으로 자연스럽게 읽히는 리뷰 문장을 구성하고, 마지막으로 해당 선택 결과를 바탕으로 대응하는 JSON 라벨을 생성한다.

문장 결합으로 인해 발생할 수 있는 부자연스러운 느낌을 방지하기 위해, 문장 첫 글자 대문자 처리, 연결 표현 사용, 그리고 혼합 감성 사례의 표현 방식에 특히 주의를 기울였다. 예를 들어 긍정과 부정이 함께 포함된 리뷰는 두 문장 구조로 구성되며, 먼저 긍정적인 평가를 서술한 뒤 “However”를 사용하여 부정적인 내용을 이어가도록 하였다.

데이터 구성에서는 단순한 데이터 양보다 카테고리, 감성 분포, 표현 방식의 다양성을 더 중요하게 고려하였다.

3.3 데이터 품질 관리

데이터 자체의 품질을 보장하기 위해 생성된 결과에 대해 두 가지 검증 절차를 적용하였다. 첫 번째는 중복 제거(deduplication)로, 최종 280 개의 리뷰 텍스트가 모두 서로 다르도록 하여 동일하거나 유사한 문장이 반복되면서 특정 표현이 암묵적으로 과도하게 학습되는 문제를 방지하였다. 두 번째는 샘플 단위 레이블 검증으로, 각 예제의 JSON 이 정상적으로 파싱되는지, 네 개의 필드가 모두 존재하는지, sentiment 값이 허용된 범주에 속하는지, score 가 유효한 정수인지, 그리고 topics 필드가 비어 있지 않은 리스트인지 등을 확인하였다. 총 280 개의 모든 샘플이 이 검증을 통과하였으며, 이를 통해 학습 이전 단계에서부터 노이즈 데이터의 개입을 차단하였다.

3.4 감성 분포

최종 데이터셋은 280 개의 서로 다른 예제로 구성되어 있으며, 감성 분포는 표 2 와 그림 1 에 제시되어 있다. 구체적으로 positive(94 개)와 negative(87 개)가 가장 높은 비중을 차지하고 있으며, neutral(33 개)은 상대적으로 적고, mixed(66 개)는 그 중간 수준을 유지한다. 이러한 분포는 의도적으로 긍정과 부정을 대체로 균형 있게 유지하면서도 neutral 과 mixed 를 일정 비

을 포함하도록 설계된 것으로, 모델이 단순히 극단적인 감성뿐만 아니라 애매하거나 혼합된 리뷰도 구분하도록 학습시키기 위한 목적을 가진다.

그림 1의 좌측 막대 그래프는 각 클래스의 절대 개수를 보여주며, 우측 도넛 차트는 전체 비율과 구성 비중을 시각적으로 나타낸다. 두 시각화를 함께 해석함으로써 데이터셋의 감성 분포 구조를 보다 직관적이고 완전하게 이해할 수 있다.

Table 2 Sentiment distribution of the training set

Sentiment class	Examples	Share
positive	94	33.6%
negative	87	31.1%
mixed	66	23.6%
neutral	33	11.8%
Total	280	100.0%

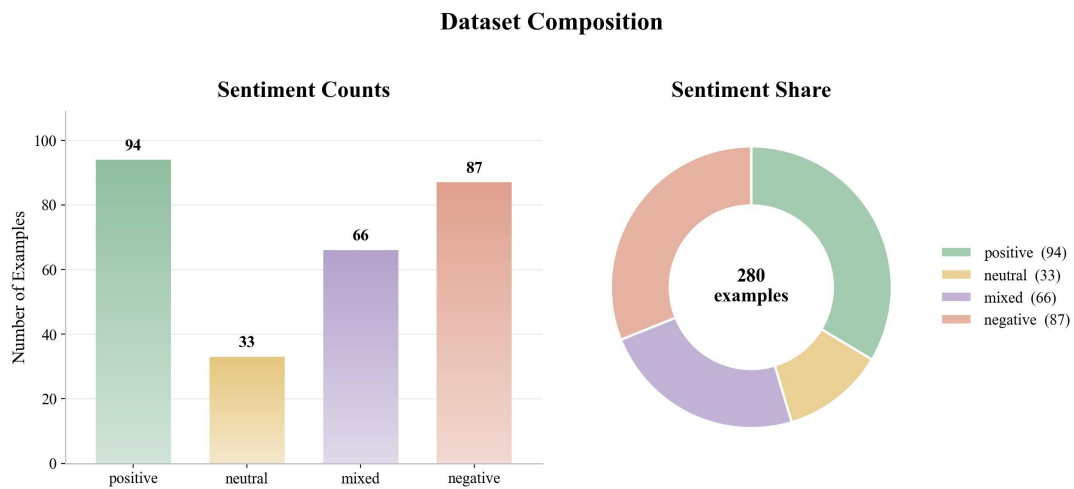


Figure 1 Sentiment counts and shares of the training set

3.5 리뷰 길이 분석

각 리뷰의 단어 수를 추가로 측정하였으며, 그 결과는 그림 2에 제시되어 있다. 리뷰 길이는 대체로 약 10 단어에서 30 단어 초반 구간에 집중되어 있으며, 평균은 약 20.9 단어이다. 최소 길이는 9 단어, 최대 길이는 37 단어로 나타났다.

이처럼 전체 분포가 짧고 비교적 밀집된 형태를 보인다는 점은 입력 데이터가 전반적으로 긴 문장을 포함하지 않음을 의미한다. 또한 이를 통해 최대 시퀀스 길이(max sequence length)를 2048로 설정한 것이 충분히 여유 있는 선택임을 확인할 수 있다. 해당 설정은 리뷰 본문과 JSON 정답을 합친 전체 길이보다 훨씬 큰 값이므로, 학습 및 추론 과정에서 길이 제한으로 인한 정보 손실이 발생하지 않는다.

그림에서 점선은 평균 단어 수를 나타내며, 함께 표시된 스무딩 곡선은 전체 길이 분포의 형태를 보다 직관적으로 보여준다.

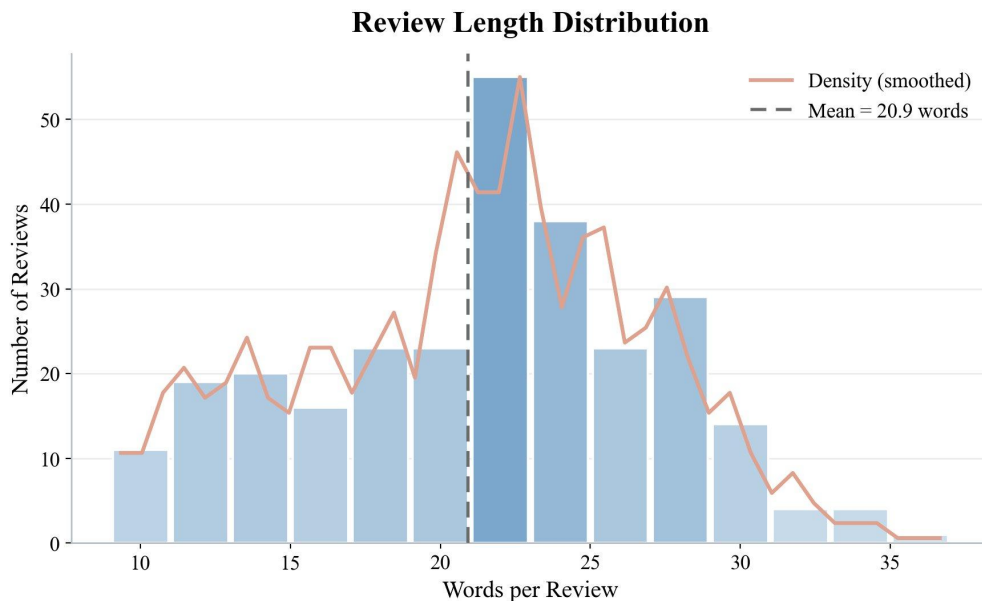


Figure 2 Distribution of review length in words

3.6 평가 항목 및 토픽 분포

모든 예제 JSON 에서 `topics` 필드를 집계하여 가장 자주 등장하는 평가 항목을 분석하였으며, 그 결과는 표 3 과 그림 3 에 제시되어 있다. 가장 높은 빈도를 보인 항목은 `price` 로 총 74 회 등장하여 가장 두드러진 비중을 차지하였다. 그 뒤를 `material`, `performance`, `design`, `value`, `comfort`, `packaging`, `battery life` 등이 잇고 있다.

이러한 분포는 비교적 타당한 결과로 볼 수 있다. `price` 는 거의 모든 제품 카테고리의 리뷰에서 공통적으로 언급될 수 있는 요소인 반면, `material`, `performance`, `comfort` 등은 특정 제품 유형에 더 밀접하게 연관된 속성이다.

또한 이 결과는 모델이 비교적 제한된 고정 어휘 집합 내에서 각 리뷰에 적합한 토픽을 선택해야 한다는 점을 보여준다. 이러한 설계는 이후 7 장에서 다루는 것처럼, 통제된 어휘 체계가 갖는 장단점 및 트레이드오프와 직접적으로 연결되는 중요한 요소이다.

Table 3 Most frequent aspects in the training set

Aspect	Freq.	Aspect	Freq.
price	74	comfort	24
material	33	packaging	22
performance	31	battery life	19
design	27	fit	16
value	25	style	16
shipping	16	durability	14

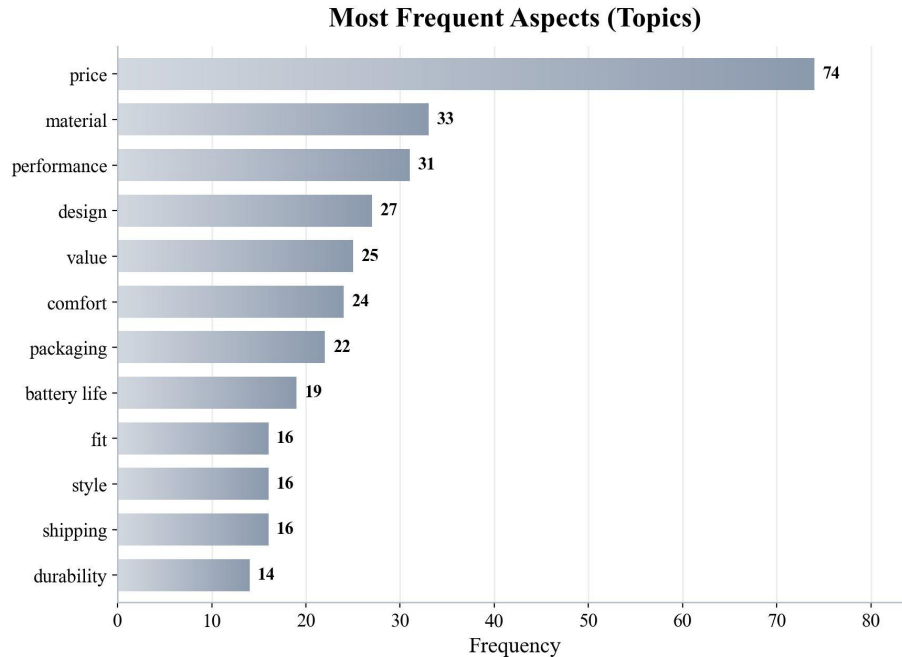


Figure 3 Twelve most frequent aspects (topics)

3.7 짧은 시스템 프롬프트: 핵심 설계 결정

여기서 별도로 설명할 만한 중요한 설계 선택은 시스템 프롬프트의 문구이다. 본 연구에서는 JSON 필드나 출력 형식을 구체적으로 설명하지 않고, 단지 “You are a customer review analysis assistant.”라는 매우 짧은 역할 정의만 제공하였다.

이 선택은 의도적인 것이다. 만약 프롬프트에서 JSON 구조와 형식을 명확하게 설명한다면, 이미 충분히 강력한 기본 모델은 단순히 프롬프트를 따라 JSON 을 출력하게 되며, 이 경우 파인튜닝의 효과를 보여줄 여지가 줄어든다.

반대로 프롬프트에 형식 힌트가 전혀 없는 경우, 기본 모델은 출력이 JSON 이어야 한다는 사실 자체를 인지하지 못하기 때문에 일반적인 대화형 응답만 생성하게 된다. 그러나 파인튜닝된 모델은 학습 과정에서 습득한 행동 패턴에 의해 동일한 조건에서도 안정적으로 올바른 JSON 을 생성할 수 있다.

이러한 설계는 이후 결과에서 나타나는 0%에서 100%로의 큰 성능 차이를 만들어내는 핵심 요인이다. 동시에 이는 파인튜닝의 본질을 보다 정확하게 보여준다. 즉, 긴 프롬프트 없이도 특정 행동을 모델의 가중치 내부에 내재화하여, 별도의 지시 없이도 일관된 출력을 생성하도록 만드는 것이다.

4 파인튜닝 방법 및 하이퍼파라미터

4.1 LoRA 및 QLoRA

LoRA의 핵심 아이디어는 선택된 선형 계층에 두 개의 저랭크(low-rank) 행렬을 추가하고, 원본 모델의 가중치는 고정된 상태로 유지하면서 이 작은 행렬들만 학습하는 것이다[1]. 이를 통해 전체 파인튜닝과 유사한 효과를 훨씬 적은 파라미터로 근사할 수 있다.

QLoRA는 이러한 구조를 기반으로 기본 모델을 4비트로 양자화하여 메모리 사용량을 더욱 줄이고, 동시에 모델 자체는 고정(freeze)한 상태로 유지하는 방식이다[2]. 본 실험에서는 기본 모델을 4비트로 로드한 뒤 완전히 고정하고, 오직 어댑터 파라미터만 학습에 참여하도록 구성하였다.

이때 학습 가능한 파라미터 수는 약 40,370,176개로 전체 모델의 약 0.5% 수준에 해당한다. 그럼에도 불구하고 모델은 안정적인 구조화 출력 능력을 학습할 수 있었으며, 이는 파라미터 효율적 파인튜닝의 핵심적인 가치라고 할 수 있다.

4.2 하이퍼파라미터 설정 및 선택 이유

어댑터 및 학습 과정에서 사용된 주요 하이퍼파라미터와 그 설정 이유는 표 4에 정리되어 있다.

LoRA rank 와 scaling factor 는 모두 16 으로 설정하였다. 이는 특정 구조화 작업에서 안정적으로 동작하는 안전한 기본값이다. dropout 은 0 으로 설정하였으며, 이는 Unsloth 에서 권장하는 최적화된 설정이다.

적용 대상 모듈은 어텐션의 4 개 프로젝션(q, k, v, o)과 피드포워드의 3 개 모듈(gate, up, down)을 포함하여 총 7 개로 구성하였다. 이를 통해 모델이 새로운 출력 구조를 학습할 충분한 표현력을 확보하도록 하였다.

학습률은 $2e-4$ 로 설정하였으며, 이는 LoRA 파인튜닝에서 일반적으로 사용되는 값이다. 학습은 총 3 에포크 동안 진행되었으며, 이는 충분한 학습 효과를 확보하면서도 소규모 고품질 데이터셋에서 과적합을 방지하기 위한 절충안이다.

배치 크기는 디바이스 기준 2 로 설정하고, gradient accumulation step 4 를 적용하여 유효 배치 크기를 8 로 구성하였다. 옵티마이저는 8-bit AdamW 를 사용하였으며[6], 이를 통해 옵티마이저 상태 메모리를 줄이면서도 AdamW 의 동작 특성을 유지하였다.

Table 4 Main LoRA/QLoRA hyperparameters and their rationale

Hyperparameter	Value	Rationale
LoRA rank r	16	Safe default for a focused task; ample capacity without bloat
lora_alpha	16	Equal to the rank; stable scaling and a common starting point
lora_dropout	0	The optimized setting recommended by Unsloth
target_modules	7 (q/k/v/o, gate/up/down)	Covers all attention and feed-forward projections

Hyperparameter	Value	Rationale
learning_rate	2e-4	A common learning rate for LoRA fine-tuning
num_train_epochs	3	Balances thorough learning against over-memorization
batch x grad-accum	2 x 4 = 8	Balances stability and efficiency within 16GB
max_seq_length	2048	Far exceeds review-plus-answer length; avoids truncation
optimizer	adamw_8bit	8-bit optimizer further saves memory [6]
lr scheduler	linear (warmup 5)	Linear decay with brief warmup for stable training
random seed	3407	Fixed seed for reproducibility

4.3 응답만을 대상으로 한 학습

학습 과정에서는 어시스턴트 응답 부분에 대해서만 손실을 계산하도록 설정하였다. 즉, 시스템 프롬프트와 사용자 리뷰는 손실 계산에 포함되지 않으며, 모델은 프롬프트와 입력을 단순히 암기하거나 그대로 복사하는 것이 아니라, 정답 JSON 을 생성하는 것 자체에만 최적화된다.

이 방식은 제한된 학습 단계에서 모든 계산 자원을 가장 중요한 목표에 집중시키는 효과가 있으며, 모델이 구조화된 출력을 더 빠르고 일관되게 학습하도록 돕는다.

4.4 학습 환경

학습은 Unsloth 프레임워크[5]를 기반으로 수행되었다. 실행 시점 기준 주요 구성 요소 버전은 Unsloth 2026.6.7, Transformers 5.5.0, PyTorch 2.12.0(CUDA 런타임), Triton 3.7.0 이며, 연산은 bfloat16 으로 수행되었다.

사용된 Chat Template 는 Qwen2.5 에 대응하는 ChatML 형식으로, 기본 모델이 사용하는 원래 입력 형식과 일치한다. 이를 통해 학습 데이터가 모델이 기대하는 입력 구조와 정확히 정렬 되도록 구성하였다.

5 학습 과정 및 모니터링

5.1 학습 구성 개요

전체 학습은 280 개의 예제를 대상으로 3 에포크 동안 수행되었으며, 유효 배치 크기는 8 이다. 총 105 step 으로 구성되었고, 해당 GPU 환경에서 약 2.9 분 만에 학습이 완료되었다. 학습 가능한 파라미터 수는 약 4,040 만 개 수준이며, 학습 과정 동안 메모리 사용량은 안정적으로 유지되어 OOM(Out of Memory) 문제는 발생하지 않았다.

학습 로그는 모든 step 에서 손실 값을 기록하도록 설정되어 이후 loss curve 분석에 필요한 완전한 데이터를 제공하였다.

5.2 손실 변화 추이

학습 손실 곡선은 그림 4 에 제시되어 있으며, 주요 구간의 값은 표 5 에 정리되어 있다. 손실 값은 첫 step 에서 약 3.35 로 시작하여 초기 약 10 여 step 동안 급격히 감소한다. 약 20 step 부근에서는 0.1 수준에 도달하고, 이후에는 완만한 저손실 구간의 평탄화 상태에 진입하여 최종적으로 약 0.01 근처에서 종료된다.

전체 step 기준 평균 손실은 0.2433 이며, 마지막 step 의 손실은 0.0140 으로 기록되었다. 이러한 곡선 형태는 모델이 매우 빠르게 목표 출력 형식을 학습했음을 보여주며, 이는 구조화 출력과 같이 비교적 고정된 패턴을 갖는 과제에서 일반적으로 기대되는 결과이다.

그림에서 얇은 선은 step 별 원시 loss 값을 나타내고, 두꺼운 선은 7-step 이동 평균을 의미한다. 아래의 그래데이션 영역은 전체적인 하락 추세를 시각적으로 강조하기 위해 추가되었다.

Table 5 Training loss at representative steps

Step	Training loss	Note
1	3.3544	first step
5	2.4562	end of warmup
10	0.4400	rapid descent
20	0.1114	entering plateau
35	0.0249	end of epoch 1
70	0.0257	end of epoch 2
105	0.0140	end of epoch 3 (last step)

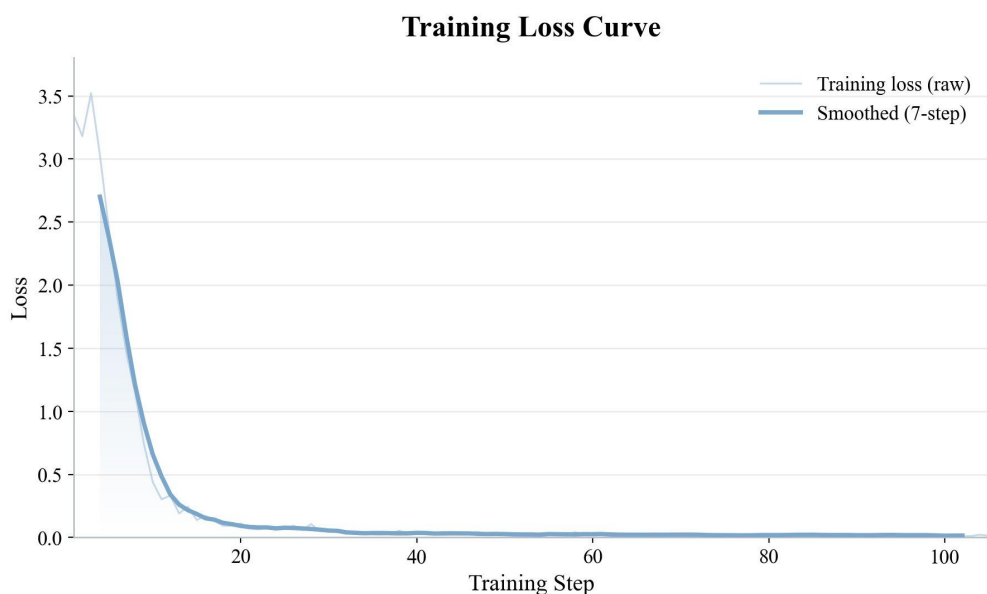


Figure 4 Training loss curve (raw and smoothed)

5.3 과적합과 과소적합 분석

과적합(overfitting)과 과소적합(underfitting)의 관점에서 보면, 본 손실 곡선은 전통적인 의미의 과소적합 상태도 아니며, 문제가 되는 과적합 상태도 아니다. 손실 값은 비교적 부드럽게 감소하여 낮은 수준에서 안정적으로 수렴하고 있으며, 이는 모델 용량과 학습률이 적절하게 설정되어 있고 학습 목표가 효과적으로 학습되었음을 의미한다.

또한 손실이 감소한 이후 다시 상승하는 형태의 변동(rebound)도 관찰되지 않기 때문에, 현재 설정된 에포크 수 범위 내에서는 학습이 안정적으로 진행되었다고 볼 수 있다.

다만 실제로 주의해야 할 부분은 소규모 데이터셋이 가지는 암기(memorization) 경향이다. 손실 값이 0에 매우 가깝게 수렴하는 현상은 모델이 일반화 능력을 획득했다기보다는 주어진 학습 샘플을 사실상 암기했을 가능성을 더 강하게 시사한다. 이러한 문제는 7장에서 한계점으로 다루며, 검증 데이터셋 도입 및 early stopping 과 같은 개선 방안을 함께 제안한다.

6 실험 결과

6.1 평가 프로토콜

파인튜닝 효과를 검증하기 위해 학습 과정에서 전혀 사용되지 않은 홀드아웃(held-out) 데이터셋을 구성하였다. 이 데이터셋은 서로 다른 카테고리 및 감성을 가진 고객 리뷰 8개와, 리뷰 분석과 무관한 일반 질문 4개로 이루어져 있다. 일반 질문에는 프랑스의 수도, 광합성의 한 문장 설명, 17과 23의 곱, 재택근무 생산성 향상 방법 등이 포함된다.

고객 리뷰에는 앞서 설명한 짧은 analyzer 프롬프트를 적용하였고, 일반 질문에는 독립적인 assistant 프롬프트를 사용하였다. 출력이 유효한 것으로 간주되기 위한 기준은 JSON 파싱 가능 여부, 네 개 필드의 존재 여부, sentiment 값의 유효성, score가 1~5 범위의 정수인지 여부,

topics 가 리스트 형태인지 여부, summary 가 문자열인지 여부이다. 이 기준은 기본 모델과 파인튜닝 모델 모두 동일하게 적용되었다.

6.2 정성적 비교: 기본 모델 vs 파인튜닝 모델

고객 리뷰에 대한 응답에서 두 모델은 매우 뚜렷한 차이를 보인다. 기본 모델은 모든 입력을 해결해야 할 일반적인 질문으로 해석하는 경향이 있어, 설명형 문단으로 응답한다. 예를 들어 누수 문제가 있는 블렌더 리뷰에 대해 “It sounds like you are experiencing a common issue with your blender's lid”와 같은 자연어 설명으로 시작하며, 이는 JSON 형식과는 전혀 다른 출력이다.

반면 파인튜닝된 모델은 8 개의 모든 리뷰에 대해 파싱 가능한 단일 JSON 형식을 생성하였다. 표 6에서는 대표적인 3 개의 리뷰를 선택하여 두 모델의 출력을 나란히 비교하였으며, 이를 통해 비정형 텍스트 응답에서 구조화된 데이터 출력으로의 변화가 매우 직관적으로 드러난다.

Table 6 Base versus fine-tuned outputs on held-out reviews (excerpts)

Review (excerpt)	Base model output (excerpt)	Fine-tuned model output
The blender ... the lid leaks every single time ...	It sounds like you are experiencing a common issue with your blender's lid ... (prose)	{"sentiment":"neutral","score":3,"topics":["ease of use"], ...}
Battery dies in about three hours and it gets ... hot while charging ...	It sounds like you are experiencing some significant issues with your device's battery ...	{"sentiment":"negative","score":2,"topics":["battery life","heat during charging"], ...}

Review (excerpt)	Base model output (excerpt)	Fine-tuned model output
Fantastic noise cancelling on my flight, and the call quality blew me away ...	It sounds like you had a very positive experience with your device or headphones ...	{"sentiment":"positive","score":5,"topics":["noise cancellation","call quality"], ...}

6.3 전체 홀드아웃 리뷰에 대한 파인튜닝 결과

선별된 사례가 아니라 전체 결과를 보여주기 위해, 표 7에는 8개의 홀드아웃 리뷰에 대한 파인튜닝 모델의 구조화된 출력이 모두 제시되어 있다. 모든 출력은 유효한 JSON 형식을 만족하며, sentiment 라벨 또한 의미적으로 타당한 값을 보인다. topics는 사전에 정의된 통제된 평가 항목(vocabulary)에서 선택되어 일관되게 구성되었다.

또한 지면상의 이유로 표에서는 생략되었지만, summary는 모두 일관된 패턴을 따르는 문장으로 생성되었다. 예를 들어 “Customer is satisfied, mainly praising noise cancellation and call quality.”와 같은 형태가 반복적으로 나타난다. 이러한 일관성은 결과를 후속 시스템에서 집계하거나 활용할 때 매우 높은 편의성을 제공한다.

Table 7 Fine-tuned structured outputs on all eight held-out reviews

#	Review (abbreviated)	Sentiment	Score	Topics	Valid
1	Blender lid leaks every time	neutral	3	ease of use	Yes
2	Backpack: pockets, comfy, survived rain	positive	5	pockets; comfort; water resistance	Yes
3	Hotel: great location, thin walls	mixed	3	location; noise	Yes
4	Battery dies in ~3h, hot	negative	2	battery life; heat during	Yes

#	Review (abbreviated)	Sentiment	Score	Topics	Valid
	charging			charging	
5	Monitor: accurate colors, wobbly stand	neutral	3	price; color accuracy; stand	Yes
6	Headphones: noise cancelling, calls	positive	5	noise cancellation; call quality	Yes
7	Shipping: 3 weeks, crushed box, intact	mixed	3	shipping; packaging	Yes
8	App: works, dated interface	neutral	3	ease of use	Yes

6.4 정량적 유효 JSON 비율

유효 JSON 생성 비율(valid-JSN rate)을 계산한 결과는 표 8 과 그림 5 에 제시되어 있다. 8 개 의 도메인 리뷰에 대해 기본 모델은 0%, 파인튜닝 모델은 100%의 성능을 보였다. 또한 4 개 의 일반 질문에 대해서는 기본 모델이 0%, 파인튜닝 모델이 25%를 기록하였다.

도메인 리뷰에서 0%에서 100%로의 성능 향상은 본 실험의 핵심 결과이며, 이는 모델이 상세한 프롬프트에 의존하지 않고도 목표 구조를 안정적으로 생성하도록 파인튜닝되었음을 직접적으로 보여준다. 그림 5 에서는 두 모델을 서로 다른 해칭 패턴으로 구분하여 표시하였으며, 각 막대에는 정확한 퍼센트 값이 함께 표기되어 있다.

Table 8 Valid-JSN output rate

Prompt type	Base model	Fine-tuned model
Domain reviews (8)	0%	100%
General questions (4)	0%	25%

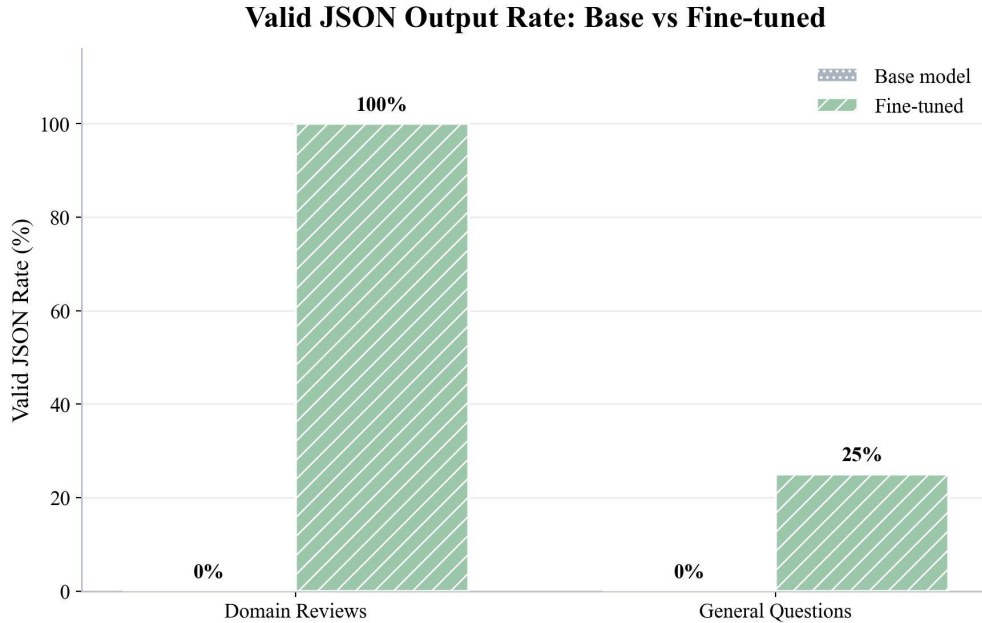


Figure 5 Valid-JSON output rate, base versus fine-tuned

6.5 망각 검증 및 형식 전이

파인튜닝 모델이 일반 질문에서 25%의 유효 JSON 비율을 보인 결과는 주의 깊게 해석할 필요가 있으며, 이는 형식 전이(format bleeding)라고 부를 수 있는 부작용을 보여준다. 표 9에는 4 개의 일반 질문에 대해 두 모델의 응답이 모두 정리되어 있다.

각 사례를 하나씩 보면, 광합성 설명 질문에서는 파인튜닝 모델이 여전히 일반적인 영어 문장으로 자연스럽게 답변하여 기본적인 언어 생성 능력은 유지되고 있음을 보여준다. 17 곱하기 23 계산 문제에서는 정답 391 을 제시했지만, 이를 본 연구의 analyzer JSON 스키마와 일치하지 않는 JSON 형태로 감싸 출력하였다. 프랑스 수도 질문에서는 `answer` 라는 키를 가진 JSON 유사 구조를 생성했으나, 이는 정의된 스키마와 일치하지 않아 유효한 결과로 간주되지 않았다. 반면 재택근무 생산성 팁 질문에서는 분석기 스키마 전체를 그대로 적용하여 JSON 을 생성했으며, 이 경우만 유효한 출력으로 인정되었다.

즉, 모델은 학습 과정에서 습득한 JSON 출력 습관을 리뷰 분석과 무관한 상황에도 일부 적용하는 경향을 보였다. 이는 형식 중심 파인튜닝에서 흔히 나타나는 과적합 유형 중 하나로, 모델이 작업의 의미적 구분보다 출력 형식 자체를 우선적으로 따르도록 학습되었음을 시사한다.

Table 9 Forgetting check on the four general questions

Question	Base answer	Fine-tuned answer	Valid
Capital of France?	The capital of France is Paris. (prose)	{"answer": "Paris", "score": 1, "topics": ["geography"], ...} (wrong key)	No
Explain photosynthesis (one sentence)	Fluent one-sentence prose	Fluent one-sentence prose (normal)	No
17 x 23 = ?	Step-by-step prose	{"role": "assistant", "content": "391"} (answer correct, odd wrapper)	No
Tip for productivity at home	Prose tip	{"sentiment": "positive", "score": 4, "topics": ["productivity"], ...}	Yes

7 토론

7.1 파인튜닝이 가져온 변화

실험 결과로부터 볼 때, 파인튜닝의 가장 직접적인 가치는 원래라면 긴 프롬프트를 통해서만 안정적으로 얻을 수 있었던 행동을 모델 내부에 고정시켰다는 점이다. 배포 환경에서는 매우 짧은 프롬프트만으로도 모델이 일관된 필드 구조의 JSON을 지속적으로 생성할 수 있

으며, 이는 후속 자동화 시스템에서 바로 활용할 수 있다는 점에서 매우 높은 실용성을 가진다.

또한 파인튜닝된 모델은 비교적 일관된 어휘 집합에서 topics 를 선택하고, summary 역시 통일된 표현 패턴을 사용하는 경향을 보인다. 이러한 일관성은 수천 개의 리뷰를 집계하여 비교 가능한 통계로 변환할 때 특히 유용하며, 각기 다른 방식으로 표현되는 기본 모델의 자유형 출력에 비해 훨씬 높은 집계 가능성을 제공한다.

중요하게 강조할 점은 이러한 가치가 모델을 “더 똑똑하게” 만드는 것이 아니라, 출력의 통제 가능성과 예측 가능성을 높인다는 것이다. 엔지니어링 시스템의 관점에서 보면, 가끔 매우 뛰어난 결과를 내지만 출력 형식이 불안정한 모델보다, 항상 일정한 구조를 반환하는 모델이 통합과 유지보수 측면에서 훨씬 더 유리하다. 일부 형식 전이(format bleeding)와 같은 결함이 존재하더라도, 목표 지향적 파인튜닝이 실제 배포 환경에서 의미를 갖는 이유가 여기에 있다.

7.2 한계

본 실험에는 몇 가지 명확한 한계가 존재한다. 첫째는 앞서 언급한 형식 전이 문제로, 모델이 JSON 출력 습관을 일반 질문에도 적용하는 현상이다. 둘째는 통제된 어휘 집합으로 인한 정보 손실이다. 일관된 출력을 위해 모든 리뷰를 고정된 aspect 단어로 매핑했기 때문에, 학습 분포를 벗어나는 리뷰에서는 과도한 단순화가 발생할 수 있다. 예를 들어 누수(leaking) 블렌더의 경우, 모델은 neutral 과 ease of use 라는 토픽을 반환했지만 실제 핵심 문제인 “누수”를 충분히 반영하지 못했다.

셋째는 summary 표현이 다소 템플릿화되어 있다는 점이다. 넷째는 데이터셋 규모의 한계로, 매우 낮은 학습 손실은 일반화 능력의 향상이라기보다는 해당 데이터셋에 대한 사실상 암기에 가깝기 때문에, 더 크고 현실적인 데이터에서의 검증이 필요하다.

7.3 개선 방향

위에서 제시된 문제들을 바탕으로 몇 가지 개선 방향을 제안할 수 있다. 형식 전이 문제를 완화하기 위해서는 일부 일반적인 질의응답 데이터를 학습에 혼합하여, 모델이 JSON을 사용할 상황과 일반 응답을 사용할 상황을 구분하도록 학습시키는 방법이 있다.

롱테일 리뷰에 대한 정확도를 높이기 위해서는 aspect 어휘 집합을 완화하거나, 리뷰 자체에서 토픽을 직접 추출하도록 설계할 수 있으며, 동시에 데이터 규모와 현실성을 확장할 필요가 있다.

또한 과적합을 보다 정확하게 평가하기 위해 독립적인 검증 데이터셋과 early stopping 을 도입할 수 있다. 추가적으로 동일한 데이터를 더 작은 모델에 적용하여 baseline 을 구성하거나, 학습률과 에포크 수에 대한 체계적인 하이퍼파라미터 탐색을 수행하면 보다 설득력 있는 결론을 얻을 수 있다.

8 모델 내보내기 및 로컬 배포

8.1 LoRA 어댑터 내보내기

학습이 완료된 이후, 먼저 경량 LoRA 어댑터를 로컬 디렉토리에 저장하였다. 이 어댑터는 크기가 작아 배포 및 재사용이 용이하며, 추론 시 기본 모델과 결합하면 파인튜닝된 동작을

그대로 복원할 수 있다. 본 실험에서는 이 과정이 안정적으로 수행되었으며, 이후 배포 단계의 기반이 되었다.

8.2 GGUF 변환 및 Ollama 활용

Ollama 와 같은 도구를 이용해 모델을 로컬에서 실행하기 위해서는 GGUF 형식에서의 변환이 필요하며, 본 실험에서는 q4_k_m 양자화를 사용하였다. 이 과정에서는 먼저 어댑터가 기본 가중치와 병합되어 전체 정밀도(full-precision) 모델이 생성되므로, 비교적 큰 용량의 가중치 파일을 다운로드하고 로컬에서 변환을 수행해야 한다. 따라서 네트워크 상태와 디스크 공간의 영향을 받는 단계이다.

GGUF 변환이 완료되면 간단한 Modelfile 과 `ollama create, ollama run` 명령만으로 로컬에서 파인튜닝된 모델을 실행하고 대화할 수 있다. 네트워크 환경의 불확실성을 고려하여, 이 내보내기 과정은 설령 실패하더라도 이미 저장된 LoRA 어댑터에는 영향을 주지 않도록 설계되었으며, 안정적인 환경에서 별도로 재시도할 수 있도록 구성하였다.

9 결론

본 프로젝트는 데이터 구축, QLoRA 파인튜닝, 학습 과정 모니터링, 기본 모델과 파인튜닝 모델의 비교, 그리고 모델 내보내기에 이르기까지 전체 파이프라인을 완성하였다. “단일 짧은 지시문만으로 구조화된 JSON 을 생성하도록 모델을 학습시키는 것”이라는 목표를 중심으로, 280 개의 다양한 예제를 구성하였으며, 단일 16GB 소비자용 GPU 에서 약 3 분 동안 학습을 수행하였다. 이 과정에서 학습 손실은 3.35 에서 0.01 까지 안정적으로 감소하였다.

홀드아웃 데이터셋 평가 결과, 도메인 리뷰에 대한 유효 JSON 생성 비율은 기본 모델의 0%에서 파인튜닝 모델의 100%로 상승하였으며, 이는 명확하고 정량적으로 확인 가능한 큰 차이를 보여준다. 동시에 형식 전이(format bleeding)와 통제된 어휘 집합으로 인한 정확도 손실과 같은 부작용도 함께 기록하고 분석하였다.

또한 이러한 한계를 개선하기 위한 방향으로 일반 질의응답 데이터의 혼합 학습, 데이터 규모 및 다양성 확장, 검증 데이터셋 도입 및 early stopping 적용 등을 제안하였다. 종합적으로 본 실험은 설정된 목표를 성공적으로 달성했을 뿐만 아니라, 파라미터 효율적 파인튜닝이 가지는 장점과 한계를 모두 구체적으로 이해할 수 있는 결과를 제공하였다.

참고문헌

- [1] Hu, E. J., Shen, Y., Wallis, P., Allen-Zhu, Z., Li, Y., Wang, S., Wang, L., & Chen, W. (2022). LoRA: Low-Rank Adaptation of Large Language Models. International Conference on Learning Representations (ICLR). arXiv:2106.09685.
- [2] Dettmers, T., Pagnoni, A., Holtzman, A., & Zettlemoyer, L. (2023). QLoRA: Efficient Finetuning of Quantized LLMs. Advances in Neural Information Processing Systems 36 (NeurIPS 2023). arXiv:2305.14314.
- [3] Qwen Team. (2025). Qwen2.5 Technical Report. arXiv:2412.15115.
- [4] Han, Z., Gao, C., Liu, J., Zhang, J., & Zhang, S. Q. (2024). Parameter-Efficient Fine-Tuning for Large Models: A Comprehensive Survey. arXiv:2403.14608.
- [5] Han, D., Han, M., & Unsloth team. (2023). Unsloth. <https://github.com/unslothai/unsloth>.
- [6] Dettmers, T., Lewis, M., Shleifer, S., & Zettlemoyer, L. (2022). 8-bit Optimizers via Block-wise Quantization. International Conference on Learning Representations (ICLR). arXiv:2110.02861.